# Projektdokumentation & Konzeption

Projekt: RezeptFinder  (Phase 1)

**Modulname**: Projekt Software Engineering (DLMCSPSE01_D)
**Name**: Daniel Faller
**Matrikelnummer**: 10807727
**Abgabedatum**: 14.10.2026

# 1. Projektübersicht & Zeitplan

Das Ziel meines Projekts ist ein einfaches Python-Programm, das bei der täglichen Entscheidung hilft: "Was koche ich heute?" Das Programm fragt nach der Kategorie (Fleisch oder ohne Fleisch) und der verfügbaren Zeit (schnell oder mehr Zeit) und schlägt ein passendes Rezept vor.

**Zeitplan:**

- **Woche 1:** Idee aufschreiben, User Stories definieren und einfache UML-Diagramme zeichnen.
- **Woche 2-4:** Python-Code schreiben (Rezept und RezeptFinder Klassen) und die Oberfläche bauen.
- **Woche 5:** Das Programm testen, Fehler beheben und die Dokumentation fertigstellen.

**Risikomanagement:**

- **Risiko 1:** Probleme bei der Nutzung von Git und GitHub.
  - *Lösung:* Ich nutze nur die grundlegenden Befehle und lade die Dateien Schritt für Schritt hoch.
- **Risiko 2:** Schwierigkeiten beim Erstellen der UML-Diagramme.
  - *Lösung:* Ich zeichne die Diagramme mit dem kostenlosen Tool Draw.io.

| **Risiko** | **Eintrittswahrscheinlichkeit** | **Schweregrad** | **Gegenmaßnahme / Lösung** |
|---|---|---|---|
| **1. Versionskonflikte bei Git/GitHub** | Mittel | Gering | Beschränkung auf grundlegende GitBefehle (commit, push); schrittweises Hochladen. |
| **Risiko** | **Eintrittswahrscheinlichkeit** | **Schweregrad** | **Gegenmaßnahme / Lösung** |

| | | | |
|---|---|---|---|
| **2. Schwierigkeiten bei UML-Diagrammen** | Gering | Mittel | Nutzung des einfachen Online-Tools Draw.io für die Erstellung. |
| **3. Logikfehler bei der Rezept-Filterung** | Mittel | Mittel | Manuelle Testläufe mit verschiedenen Kombinationen von Kategorien und Zeiten. |

# 2. Anforderungsdokument (User Stories)

- **Zielgruppe:** Personen, die schnell eine einfache Entscheidungshilfe für das Kochen suchen.
- **Entwickler:** Ein Einsteiger, der die Grundkonzepte der objektorientierten Programmierung anwendet.

**User Stories:**

- **US1:** Als Benutzer möchte ich auswählen können, ob ich Fleisch oder vegetarisch essen möchte.
- **US2:** Als Benutzer möchte ich angeben, ob es schnell gehen muss oder ob ich Zeit habe.
- **US3:** Als Benutzer möchte ich das Rezept sehen und am Ende entscheiden können, ob ich noch ein anderes Rezept suchen möchte.

**Nicht-funktionale Anforderungen:**

- Die Benutzeroberfläche soll übersichtlich und einfach zu bedienen sein.
- Das Programm soll stabil laufen und bei falschen Eingaben nicht einfach abstürzen.

**Glossar:**

- **GUI (Graphical User Interface):** Grafische Benutzeroberfläche zur Bedienung mit der Maus.

- **OOP (Objektorientierte Programmierung):** Strukturierung des Codes in Klassen und Objekte.
- **Tkinter:** Standard-Bibliothek in Python für grafische Fenster.

# 3. Spezifikationsdokument & Datenmodell

- **Datenmodell:** Es gibt eine Klasse Rezept für die einzelnen Gerichte (mit Namen, Kategorie,

  Zeit und Rezepttext) und eine Klasse RezeptFinder, welche die Rezepte speichert und filtert.
- **Geschäftsregel:** Ein Rezept wird nur vorgeschlagen, wenn Kategorie und Zeit genau zur Auswahl des Benutzers passen.
- **Benutzeroberfläche:** Ein einfaches Tkinter-Fenster mit Radiobuttons für die Auswahl und Klick-Buttons für die Steuerung.

**System- und Benutzerschnittstellen (Mock-up & Validierung)**

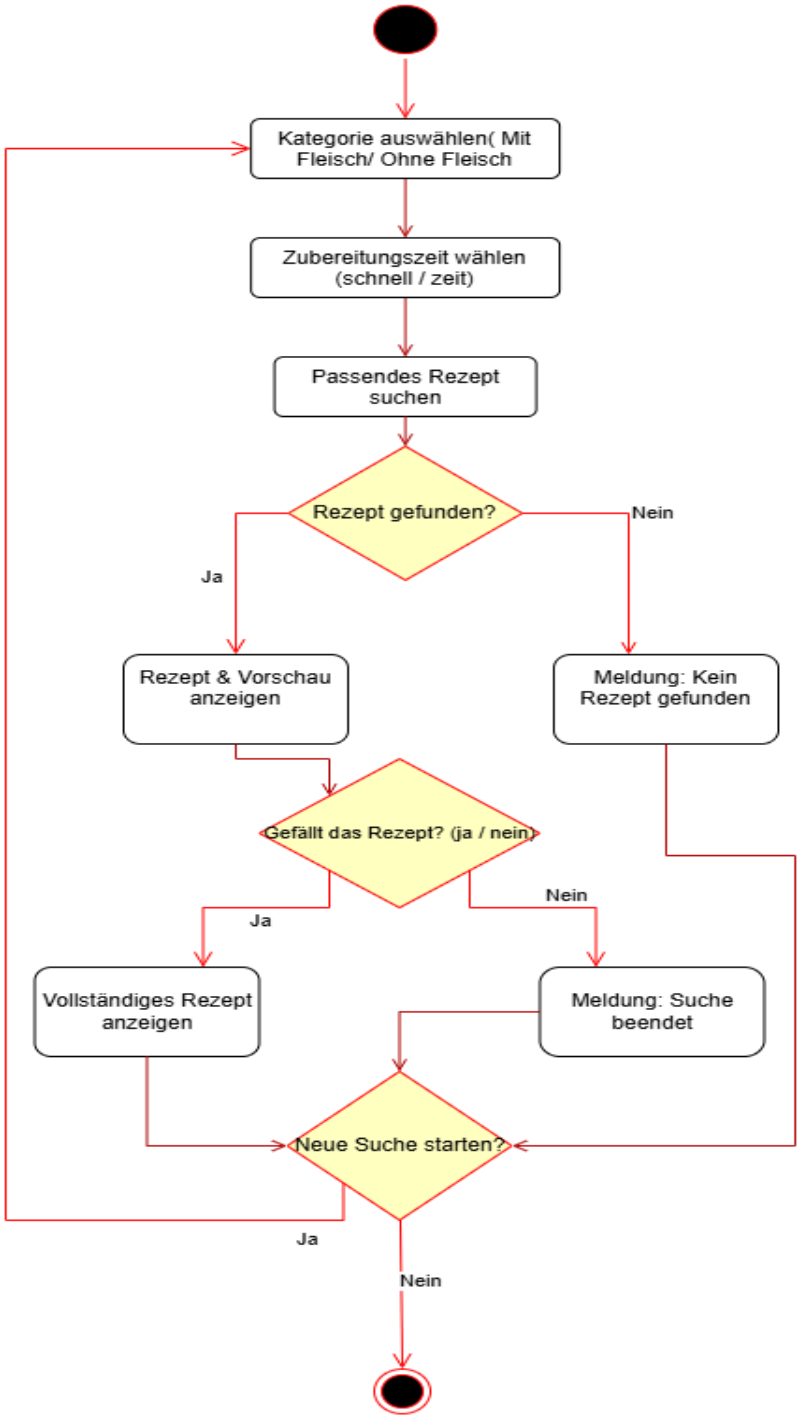
- **Systemschnittstellen:** Da es sich um eine lokale Desktop-Anwendung handelt, gibt es keine komplexen Webschnittstellen. Die System-Schnittstellen beschränken sich auf die Python-internen Standardmodule (tkinter für die GUI, random für die Zufallsauswahl) sowie das lokale Betriebssystem.
- **Benutzerschnittstellen & Dialogfluss (Mock-up):** Das Programm besitzt eine klare, übersichtliche Dialogstruktur im Tkinter-Fenster:




- **Eingabevalidierung:** Wenn der Benutzer auf den Such-Button klickt, ohne eine Auswahl getroffen zu haben, fängt das Programm diesen Zustand im Code ab, verhindert einen Absturz und gibt stattdessen einen Hinweis im Ausgabefenster aus.

# 4. Interaktionsablauf

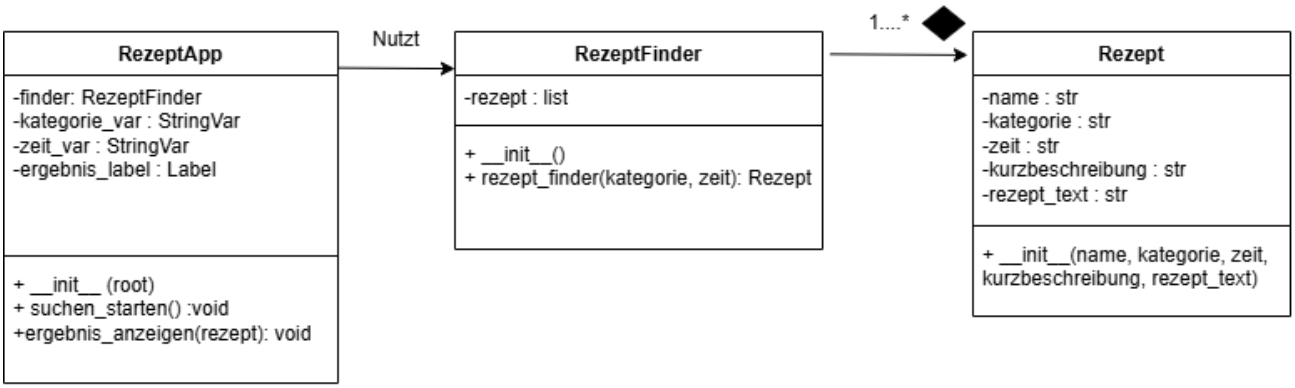
**Aktivitätsdiagramm**




Beschreibt den logischen Ablauf im Programm: Nach der Auswahl von Kategorie und Zeit prüft das System, ob ein passendes Rezept vorhanden ist, zeigt das Ergebnis an und gibt dem Benutzer die Möglichkeit, die Suche bei Bedarf zu wiederholen

### Klassendiagramm

Stellt die objektorientierte Struktur des Programms dar. Die RezeptApp steuert die Benutzeroberfläche, der RezeptFinder filtert die Gerichte, und die Klasse Rezept speichert die konkreten Daten der einzelnen Rezepte.




### UML-Use-Case-Diagramm

Zeigt den typischen Ablauf aus Sicht des Benutzers: Er wählt die Kategorie und die Zeit aus, startet die Suche, lässt sich das Rezept anzeigen oder beginnt auf Wunsch eine neue Suche.

